

# Automation testing

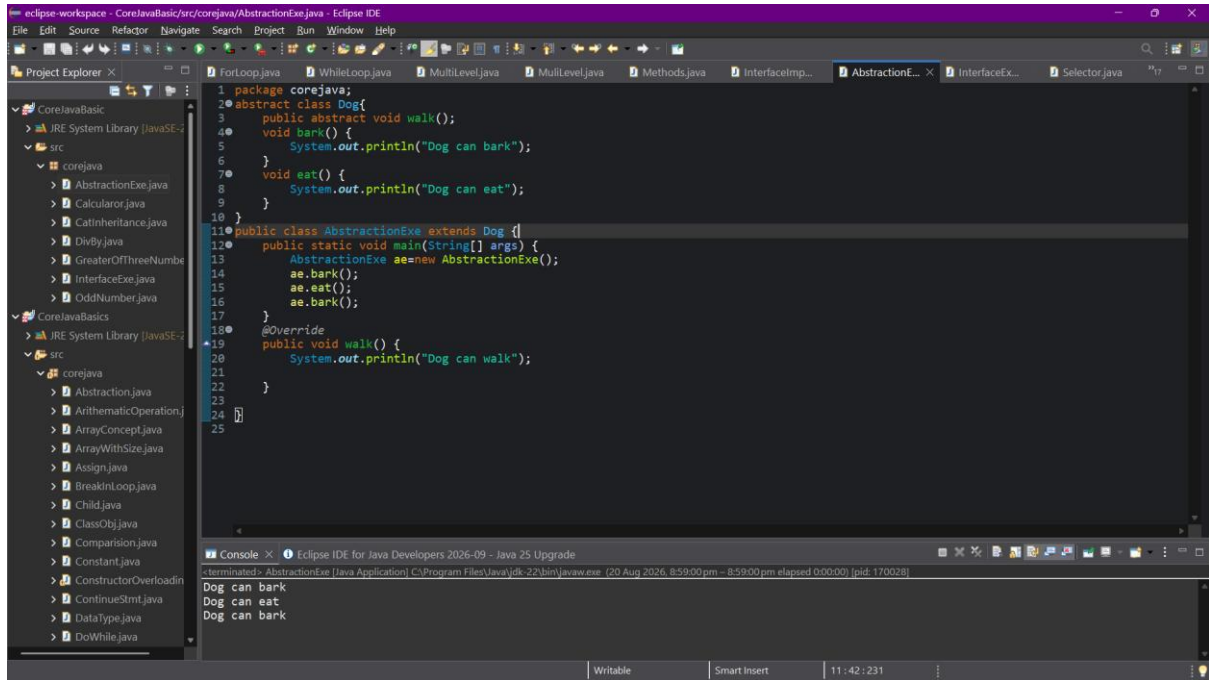
## Day-05 Assignment

1.Explain the Difference between Abstract Class and Interface with example programs

| Abstract Class                                    | Interface                                                                  |
|---------------------------------------------------|----------------------------------------------------------------------------|
| Declared using abstract class                     | Declared using interface                                                   |
| Class uses extends                                | Class uses implements                                                      |
| Can have abstract and concrete methods            | Mainly used to define a contract; can also have default and static methods |
| Can have instance variables                       | Variables are implicitly public static final                               |
| Can have constructors                             | Cannot have constructors                                                   |
| A class can extend only <b>one</b> abstract class | A class can implement <b>multiple interfaces</b>                           |
| Can have different access modifiers for methods   | Abstract interface methods are implicitly public                           |
| Used when classes have a common base/relationship | Used when unrelated classes need to follow the same behavior               |

# Example Program:

## Abstract Class:



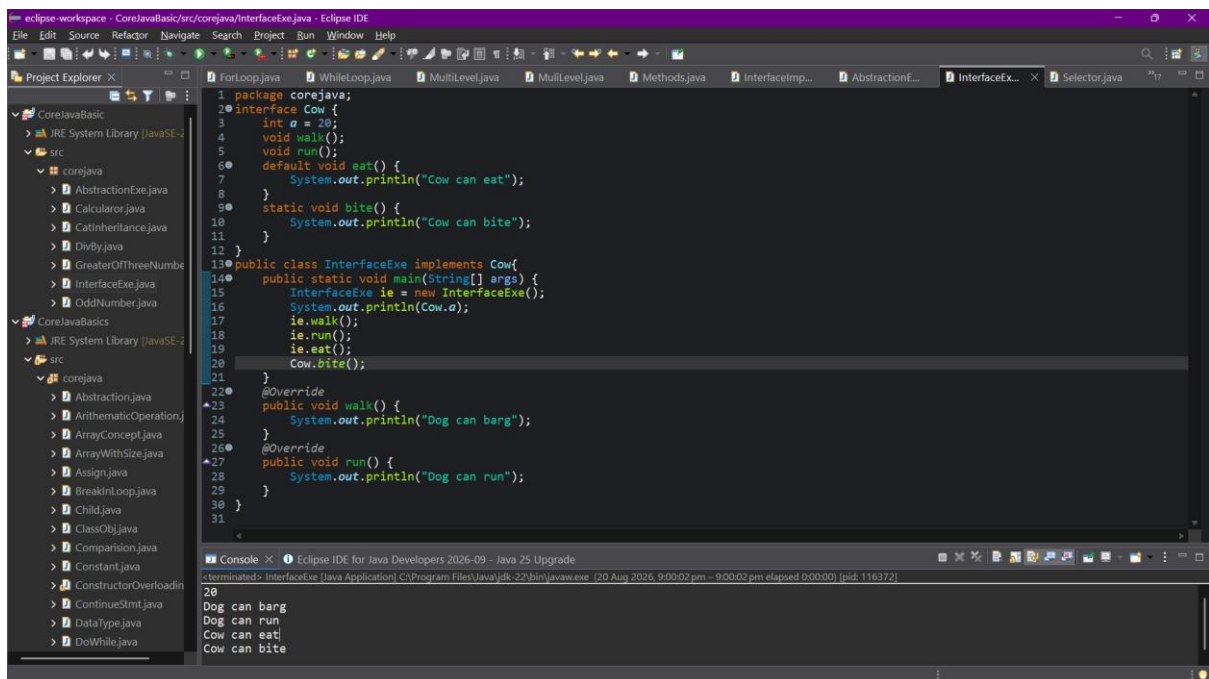
The screenshot shows the Eclipse IDE with a project named 'CoreJavaBasic'. The 'Project Explorer' on the left shows the project structure. The main editor displays the code for 'AbstractionExe.java'. The code defines an abstract class 'Dog' with methods 'walk()', 'bark()', and 'eat()'. The 'AbstractionExe' class extends 'Dog' and implements these methods. The console output shows the results of the program execution.

```
1 package corejava;
2 abstract class Dog{
3     public abstract void walk();
4     void bark() {
5         System.out.println("Dog can bark");
6     }
7     void eat() {
8         System.out.println("Dog can eat");
9     }
10 }
11 public class AbstractionExe extends Dog {
12     public static void main(String[] args) {
13         AbstractionExe ae=new AbstractionExe();
14         ae.bark();
15         ae.eat();
16         ae.bark();
17     }
18     @Override
19     public void walk() {
20         System.out.println("Dog can walk");
21     }
22 }
23
24
25
```

Console Output:

```
Terminated: AbstractionExe [Java Application] C:\Program Files\Java\jdk-22\bin\java.exe (20 Aug 2026, 8:59:00 pm - 8:59:00 pm elapsed 0:00:00) [pid: 170028]
Dog can bark
Dog can eat
Dog can bark
```

## Interface Class:



The screenshot shows the Eclipse IDE with a project named 'CoreJavaBasic'. The 'Project Explorer' on the left shows the project structure. The main editor displays the code for 'InterfaceExe.java'. The code defines an interface 'Cow' with methods 'walk()', 'run()', and 'eat()'. The 'InterfaceExe' class implements these methods. The console output shows the results of the program execution.

```
1 package corejava;
2 interface Cow {
3     int a = 20;
4     void walk();
5     void run();
6     default void eat() {
7         System.out.println("Cow can eat");
8     }
9     static void bite() {
10         System.out.println("Cow can bite");
11     }
12 }
13 public class InterfaceExe implements Cow{
14     public static void main(String[] args) {
15         InterfaceExe ie = new InterfaceExe();
16         System.out.println(Cow.a);
17         ie.walk();
18         ie.run();
19         ie.eat();
20         Cow.bite();
21     }
22     @Override
23     public void walk() {
24         System.out.println("Dog can barg");
25     }
26     @Override
27     public void run() {
28         System.out.println("Dog can run");
29     }
30 }
31
```

Console Output:

```
Terminated: InterfaceExe [Java Application] C:\Program Files\Java\jdk-22\bin\java.exe (20 Aug 2026, 9:00:02 pm - 9:00:02 pm elapsed 0:00:00) [pid: 116372]
20
Dog can barg
Dog can run
Cow can eat
Cow can bite
```

2.Explain about Overloading and Overriding with example programs:

### **Method Overloading:**

**Method Overloading** means creating multiple methods with the **same name but different parameters** in the same class.

The parameters can differ in:

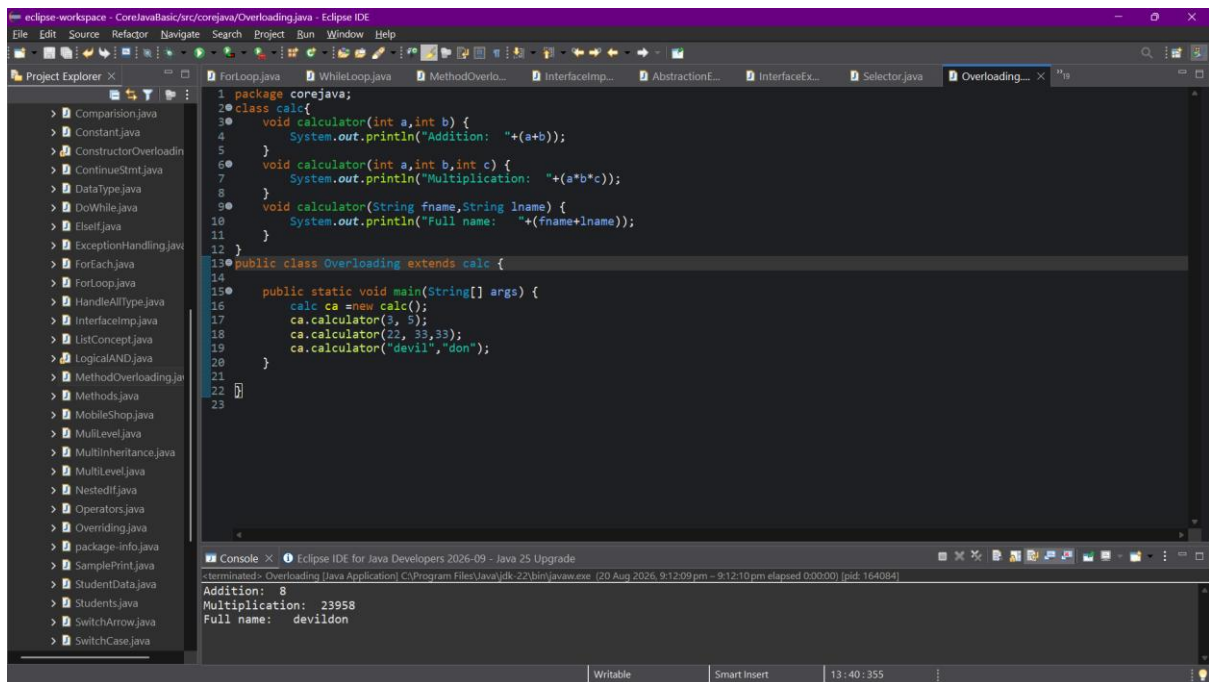
- Number of parameters
- Data type of parameters
- Order of parameters

### **2.Method Overriding:**

**Method Overriding** occurs when a **child class provides its own implementation** of a method that is already present in the parent class.

The method name and parameters must be the same.

# Program for method overloading:



```
1 package corejava;
2 class calc{
3     void calculator(int a,int b) {
4         System.out.println("Addition:  "+(a+b));
5     }
6     void calculator(int a,int b,int c) {
7         System.out.println("Multiplication:  "+(a*b*c));
8     }
9     void calculator(String fname,String lname) {
10        System.out.println("Full name:  "+(fname+lname));
11    }
12 }
13 public class Overloading extends calc {
14
15     public static void main(String[] args) {
16         calc ca =new calc();
17         ca.calculator(3, 5);
18         ca.calculator(22, 33,33);
19         ca.calculator("devil","don");
20     }
21 }
22
23
```

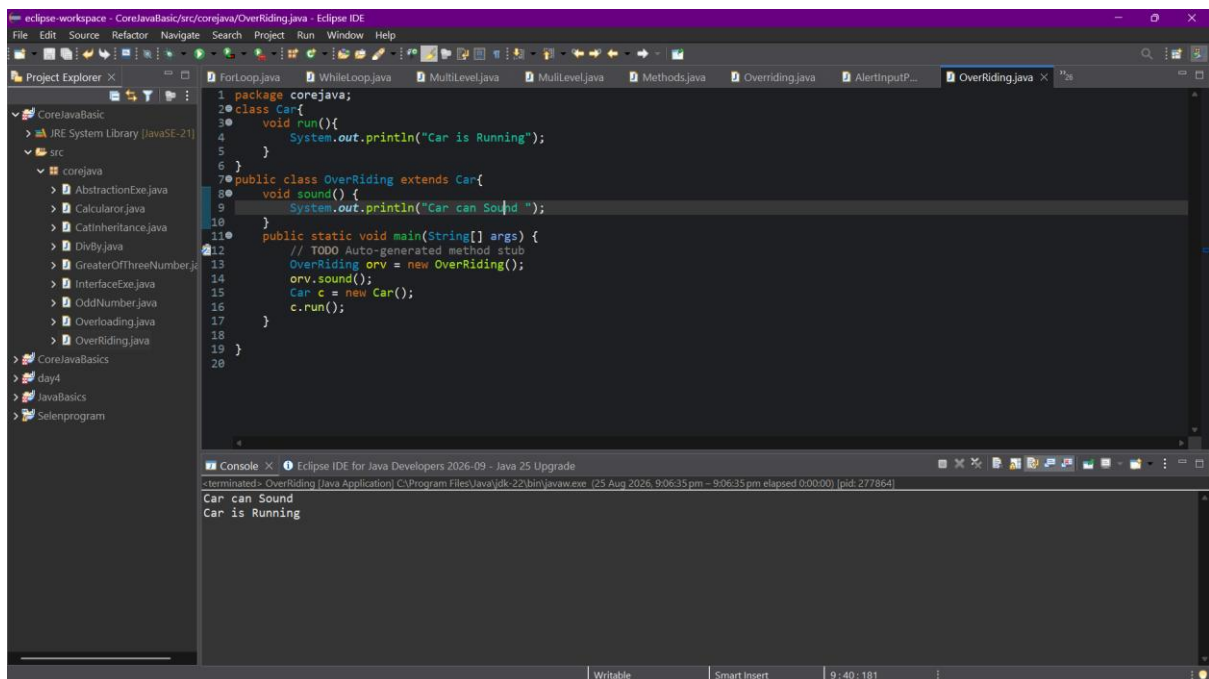
Console Output:

```

Addition:  8
Multiplication:  23958
Full name:  devildon

```

# Program for Method Overriding:



```
1 package corejava;
2 class Car{
3     void run(){
4         System.out.println("Car is Running");
5     }
6 }
7 public class OverRiding extends Car{
8     void sound() {
9         System.out.println("Car can Sound ");
10    }
11 }
12 public static void main(String[] args) {
13     // TODO Auto-generated method stub
14     OverRiding orv = new OverRiding();
15     orv.sound();
16     Car c = new Car();
17     c.run();
18 }
19
20
```

Console Output:

```

Car can Sound
Car is Running

```